EN3160: Image Processing and Machine Vision

Assignment 3: Convolutional Neural Networks

Department of Electronic and Telecommunication Engineering

University of Moratuwa

October 19, 2026

Name: Samarasinghe S.M.R.R.
Index Number: 230566U
GitHub Repository: github.com/RumethRanhinda/EN3160-Vision-UoM

---

1. Task 1

1.1 Problem Description and Network Architecture

In this task, we extend the single-layer dense network provided in Listing 1 of the assignment by implementing a two-layer neural network with manually computed forward and backward propagations (without relying on PyTorch autograd). The network is trained on the CIFAR-10 dataset to classify 32×32 RGB images into 10 distinct classes.

The required modifications are:

1. **Middle Layer with Non-Linearity:** Introducing a hidden layer with $H = 100$ nodes followed by a Sigmoid activation function $\sigma(z) = \frac{1}{1+e^{-z}}$.
2. **Cross-Entropy Loss:** Replacing the Mean Squared Error (MSE) loss with Softmax Cross-Entropy loss.
3. **Training and Evaluation:** Training the network for 10 epochs and reporting both training and test classification accuracies along with the loss trajectory.

Mathematical Formulation:

- **Input:** Batch of N flattened CIFAR-10 images $X \in \mathbb{R}^{N \times D_{in}}$, where $D_{in} = 3 \times 32 \times 32 = 3072$.
- **Hidden Layer (Linear + Sigmoid):**

$$z_1 = XW_1 + b_1 \in \mathbb{R}^{N \times 100}$$
$$a_1 = \sigma(z_1) = \frac{1}{1 + e^{-z_1}} \in \mathbb{R}^{N \times 100}$$

  where $W_1 \in \mathbb{R}^{3072 \times 100}$ and $b_1 \in \mathbb{R}^{100}$.

- **Output Layer (Linear):**

$$z_2 = a_1 W_2 + b_2 \in \mathbb{R}^{N \times 10}$$

  where $W_2 \in \mathbb{R}^{100 \times 10}$ and $b_2 \in \mathbb{R}^{10}$.

- **Softmax and Cross-Entropy Loss with Regularization:** The class probabilities $P \in \mathbb{R}^{N\times 10}$ and batch loss $\mathcal{L}$ are given by:

$$P_{ik} = \frac{e^{z_{2,ik}}}{\sum_{j=1}^{K} e^{z_{2,ij}}}$$

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log(P_{i,y_i}) + \lambda\left(\sum W_1^2 + \sum W_2^2\right)$$

where y_i is the ground-truth class index for sample i.

Analytical Gradient Derivations (Backpropagation):

1. **Gradient w.r.t. Output Logits (z_2):**

$$\frac{\partial \mathcal{L}}{\partial z_2} = \frac{1}{N}(P - Y_{\text{one-hot}}) \in \mathbb{R}^{N\times 10}$$

2. **Gradients w.r.t. Layer 2 Parameters (W_2, b_2):**

$$\frac{\partial \mathcal{L}}{\partial W_2} = a_1^T \frac{\partial \mathcal{L}}{\partial z_2} + 2\lambda W_2, \quad \frac{\partial \mathcal{L}}{\partial b_2} = \sum_{i=1}^{N}\left(\frac{\partial \mathcal{L}}{\partial z_2}\right)_i$$

3. **Gradient w.r.t. Hidden Activations (a_1):**

$$\frac{\partial \mathcal{L}}{\partial a_1} = \frac{\partial \mathcal{L}}{\partial z_2} W_2^T \in \mathbb{R}^{N\times 100}$$

4. **Gradient w.r.t. Hidden Pre-activations (z_1):**

$$\frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial a_1} \odot \sigma'(z_1) = \frac{\partial \mathcal{L}}{\partial a_1} \odot a_1 \odot (1 - a_1) \in \mathbb{R}^{N\times 100}$$

5. **Gradients w.r.t. Layer 1 Parameters (W_1, b_1):**

$$\frac{\partial \mathcal{L}}{\partial W_1} = X^T \frac{\partial \mathcal{L}}{\partial z_1} + 2\lambda W_1, \quad \frac{\partial \mathcal{L}}{\partial b_1} = \sum_{i=1}^{N}\left(\frac{\partial \mathcal{L}}{\partial z_1}\right)_i$$

[ ]:
```python
# 1. Dataloading and Preprocessing
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# Data transformations: Convert images to tensors and normalize pixel values to [-1, 1]
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

```
batch_size = 50

# Load CIFAR-10 training set
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                          shuffle=True, num_workers=2)

# Load CIFAR-10 test set
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                         shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')

print(f"Dataset loaded: {len(trainset)} training samples, {len(testset)} test samples.")
```

1.2 Network Parameters and Hyperparameters

We define a two-layer architecture with $D_{in} = 3072$ input nodes, a middle hidden layer of $H = 100$ nodes with sigmoid activation, and $K = 10$ output class nodes. Weights are randomly initialized from a normal distribution scaled by std $= 10^{-3}$, and biases are initialized to zero. Training hyperparameters are set to $\eta = 0.01$ (learning rate), $\gamma = 0.95$ (decay factor), and $\lambda = 10^{-4}$ (L2 regularization).

[ ]:
```
# 2. Define Network Parameters and Hyperparameters
Din = 3 * 32 * 32   # 3072 input nodes (flattened 32x32x3 CIFAR-10 image)
H = 100             # 100 hidden layer nodes (Requirement 1a)
K = 10              # 10 output classes
std = 1e-3          # Standard deviation for weight initialization

# Layer 1 parameters (Input -> Hidden)
w1 = torch.randn(Din, H) * std
b1 = torch.zeros(H)

# Layer 2 parameters (Hidden -> Output)
w2 = torch.randn(H, K) * std
b2 = torch.zeros(K)

# Hyperparameters
epochs = 10         # 10 epochs (Requirement 1c)
lr = 0.01           # Initial learning rate
lr_decay = 0.95     # Decay factor applied after each epoch
reg = 1e-4          # L2 Regularization coefficient
loss_history = []

print(f"Network parameters initialized: w1={w1.shape}, b1={b1.shape}, w2={w2.shape}, b2={b2.
↪shape}")
```

1.3 Training Loop with Manual Forward and Backward Propagation

The network is trained for 10 epochs. In every iteration, we perform:

1. **Forward pass:** Compute $z_1 = XW_1 + b_1$, $a_1 = \sigma(z_1)$, $z_2 = a_1W_2 + b_2$, and class probabilities $P = \text{softmax}(z_2)$.

2. **Loss calculation:** Cross-entropy loss $\mathcal{L} = -\frac{1}{N}\sum \log(P_{i,y_i}) + \text{reg}$.
3. **Backpropagation:** Compute gradients dz_2, dW_2, db_2, da_1, dz_1, dW_1, db_1 analytically.
4. **Parameter update:** Update parameters using gradient descent: $W \leftarrow W - \eta\, dW$, $b \leftarrow b - \eta\, db$.

[ ]:
```
# 3. Training Loop
for epoch in range(epochs):
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # Get inputs and labels
        inputs, labels = data
        Ntr = inputs.shape[0]                          # Batch size (50)
        x_train = inputs.view(Ntr, -1)                 # Flatten input to (Ntr, Din)
        y_train_onehot = nn.functional.one_hot(labels, K).float()  # (Ntr, K)

        # --- Forward pass ---
        # Hidden layer with sigmoid activation (Requirement 1a)
        z1 = x_train.mm(w1) + b1
        a1 = torch.sigmoid(z1)

        # Output layer logits
        z2 = a1.mm(w2) + b2

        # Softmax probabilities
        probs = torch.softmax(z2, dim=1)

        # Cross-Entropy Loss with L2 Regularization (Requirement 1b)
        loss = -torch.sum(torch.log(probs[range(Ntr), labels] + 1e-12)) / Ntr \
               + reg * (torch.sum(w1 ** 2) + torch.sum(w2 ** 2))
        loss_history.append(loss.item())
        running_loss += loss.item()

        # --- Backpropagation ---
        # Gradient of cross-entropy loss w.r.t. output logits z2
        dz2 = (probs - y_train_onehot) / Ntr

        # Gradients w.r.t. Layer 2 parameters
        dw2 = a1.t().mm(dz2) + 2.0 * reg * w2
        db2 = dz2.sum(dim=0)

        # Backpropagate through hidden layer and sigmoid activation
        da1 = dz2.mm(w2.t())
        dz1 = da1 * a1 * (1.0 - a1)    # d(sigmoid)/dz = a * (1 - a)

        # Gradients w.r.t. Layer 1 parameters
        dw1 = x_train.t().mm(dz1) + 2.0 * reg * w1
        db1 = dz1.sum(dim=0)

        # --- Parameter update ---
        w1 -= lr * dw1
        b1 -= lr * db1
        w2 -= lr * dw2
        b2 -= lr * db2

    # Print epoch summary
    epoch_loss = running_loss / len(trainloader)
```

```
    print(f"Epoch {epoch + 1:2d}/{epochs}, Loss: {epoch_loss:.4f}, Learning Rate: {lr:.6f}")

    # Decay learning rate
    lr *= lr_decay
```

1.4 Loss Trajectory and Accuracy Evaluation

We plot the loss trajectory over iterations to verify network convergence, followed by calculating the classification accuracy on both the training set and the test set.

[ ]:
```
# 4. Plotting the Loss History
plt.figure(figsize=(8, 4))
plt.plot(loss_history, color='#55499C', linewidth=1.2)
plt.title("Task 1: Training Loss History (2-Layer Network with Cross-Entropy)")
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

# 5. Calculate Accuracy on Training Set
correct_train = 0
total_train = 0
with torch.no_grad():
    for data in trainloader:
        inputs, labels = data
        Ntr = inputs.shape[0]
        x_train = inputs.view(Ntr, -1)

        # Forward pass
        z1 = x_train.mm(w1) + b1
        a1 = torch.sigmoid(z1)
        z2 = a1.mm(w2) + b2

        predicted_train = torch.argmax(z2, dim=1)
        total_train += labels.size(0)
        correct_train += (predicted_train == labels).sum().item()

train_acc = 100.0 * correct_train / total_train
print(f"Final Training Accuracy: {train_acc:.2f}%")

# 6. Calculate Accuracy on Test Set
correct_test = 0
total_test = 0
with torch.no_grad():
    for data in testloader:
        inputs, labels = data
        Nte = inputs.shape[0]
        x_test = inputs.view(Nte, -1)

        # Forward pass
        z1 = x_test.mm(w1) + b1
        a1 = torch.sigmoid(z1)
        z2 = a1.mm(w2) + b2

        predicted_test = torch.argmax(z2, dim=1)
        total_test += labels.size(0)
        correct_test += (predicted_test == labels).sum().item()
```

```
test_acc = 100.0 * correct_test / total_test
print(f"Final Test Accuracy:     {test_acc:.2f}%")
```